

Mini Software

Conversión de Unidades

05 de julio de 2026
Introducción a los Repositorios de
Código Distribuido

Descripción

El proyecto consiste en el desarrollo de un mini software para la conversión de unidades, implementado usando HTML, CSS y JavaScript. La aplicación permite ingresar un valor numérico, seleccionar la unidad origen (lo que deseamos convertir) y la unidad destino (a lo que queremos convertir), para después ser calculado y mostrado el resultado de la conversión. El sistema incorpora distintas categorías de conversión, entre las que se pueden observar están, la longitud, masa, volumen, temperatura, tiempo, ángulo, datos, frecuencia, fuerza, entre otros. Durante el desarrollo se utilizó Git como sistema de control de versiones y GitHub como repositorio remoto, aplicando un desarrollo mediante commits, ramas de trabajo, integración de cambios y pruebas, con el objetivo de mantener un historial organizado y facilitar la evolución del software.

Estructura

El proyecto está organizado en carpetas y archivos principales para separar la lógica, la interfaz y las pruebas.

- `conversor.html`: Es el archivo principal de la interfaz. Contiene la estructura del conversor y conecta los archivos CSS y JavaScript.
- `Style/conversor.css`: Este archivo contiene los estilos visuales de la aplicación, como colores, tamaños, botones, formularios y diseño general.
- `Src/softwaremini.js`: Este controla la interacción con el usuario. Obtiene los datos ingresados en la pantalla, llama a las funciones de conversión y muestra el resultado.
- `Src/conversiones.js`: Posee las funciones y fórmulas necesarias para realizar las conversiones entre las diferentes unidades.
- `Src/unidades.js`: Almacena las unidades disponibles y sus factores de conversión, organizados por categoría como longitud, masa, tiempo, temperatura, área, volumen, velocidad, presión, energía, potencia, fuerza, datos, corriente, voltaje, resistencia, ángulo y consumo de combustible.
- `Test/pruebas.js`: contiene pruebas básicas para verificar que algunas conversiones funcionen correctamente.
- `README.md`: documento principal del proyecto. Incluye información general, instrucciones de uso y detalles de la estructura del repositorio.

Evolución

El desarrollo del proyecto se realizó de forma progresiva usando Git y distintas ramas de trabajo.

1. Primero se creó una versión base del proyecto con los archivos iniciales del mini software de conversiones. Esta etapa permitió tener una estructura mínima para comenzar el desarrollo.
2. Luego se agregó un primer código HTML para visualizar el conversor en pantalla. En esta versión inicial se empezó a construir la interfaz básica del usuario.
3. Después se creó la rama feature-conversiones, donde se trabajó la lógica relacionada con las conversiones. En esta rama se agregaron validaciones, como impedir valores negativos en unidades donde no corresponde.
4. Más adelante se agregaron archivos y funciones para organizar mejor la lógica del programa, separando las unidades disponibles y las fórmulas de conversión.
5. También se desarrolló la rama feature-interfaz, enfocada en mejorar la presentación visual del conversor. En esta parte se agregaron estilos, colores, bordes, ubicación de elementos y mejoras generales en la interfaz.
6. Se creó una rama test para incorporar pruebas básicas de validación y comprobar que algunas conversiones funcionaran correctamente.
7. Luego se incorporó la conversión de monedas al proyecto. Para esto se agregó una categoría de monedas dentro de las unidades disponibles, incluyendo valores como USD, CLP, EUR y otros códigos de monedas.
8. Después se actualizó la interfaz para que el usuario no mezclara las unidades con las monedas. Se agregó una primera selección donde se debe escoger si se quiere convertir una unidad o una moneda.
9. Cuando el usuario escoge convertir una unidad, se agregó una segunda selección para elegir la categoría correspondiente, como longitud, masa, volumen, temperatura, tiempo, área, velocidad, entre otras categorías.
10. Cuando el usuario escoge convertir moneda, el selector de categoría se oculta, ya que no corresponde mostrar opciones como masa o longitud en este caso. Así la interfaz es más clara y fácil de usar.
11. También se mejoró la forma en que se muestran las monedas, agregando una descripción junto al código. Por ejemplo, en vez de mostrar solo CLP, ahora puede aparecer como CLP - peso chileno, lo que ayuda a conocer los nombres de todos los códigos de monedas.
12. Por último, se fue integrando, mediante merges, el trabajo de las distintas ramas secundarias (feature-conversiones, feature-interfaz y test) hacia la rama principal main, dejando el proyecto completo, ordenado y funcional. Con esto, el conversor pasó de ser una versión básica a un mini software con interfaz, lógica, validaciones, pruebas y capacidad para realizar distintos tipos de conversiones.

Análisis del historial de git

Con un total de 30 commits sin contar con los respectivos a este documento, se finalizó el desarrollo del mini software. La salida del comando `git log --pretty=format:"%h | %ad | %an | %s" --date=short` es

```
b1b2c5b | 2026-07-01 | Diego Castro Satt | explicacion de ejecucion de pruebas
bf6032c | 2026-07-01 | Diego Castro Satt | Separacion de conversiones por tipo y categoria
c5d39f5 | 2026-07-01 | Diego Castro Satt | sin querer meti un enter en el gitignore
e041607 | 2026-06-30 | Diego Castro Satt | agrego segunda opcion para descargar el repo
2acab8b | 2026-06-30 | Diego Castro Satt | merge de main con rama test
0fe81ea | 2026-06-30 | Diego Castro Satt | Lógica de conversión y formato de resultados
3c358f8 | 2026-06-29 | Diego Castro Satt | merge de feature-interfaz y test
ccb72f1 | 2026-06-29 | Diego Castro Satt | simplificacion de pruebas, se reduce el numero de funciones en js
83c3fe4 | 2026-06-29 | Diego Castro Satt | cambio de logica para que las pruebas entreguen true si la conversion es
correcta y false si no lo es, y agrego mas pruebas
94a8202 | 2026-06-29 | Diego Castro Satt | cambio la prueba antigua a una mas adoc
b4e3d72 | 2026-06-28 | Diego Castro Satt | conflicto del merge solucionado
7f61665 | 2026-06-28 | Diego Castro Satt | agrego funcionalidades, ramas y ejecucion al readme
9f8cf04 | 2026-06-28 | Diego Castro Satt | agrego funcionalidades, ramas y ejecucion al readme
6992e29 | 2026-06-28 | Diego Castro Satt | agrego funcion para calculo de moneda respecto al dolar
acf9217 | 2026-06-28 | Diego Castro Satt | cambio de consumo de bencina a monedas de todos los paises
0bcc50d | 2026-06-28 | Diego Castro Satt | cambio de consumo de bencina a monedas de todos los paises
d4d99d3 | 2026-06-28 | Diego Castro Satt | README con titulo, funcionalidades del software y como ejecutarlo
864938a | 2026-06-26 | Diego Castro Satt | diseno (colores, ubicacion, bordes...) de la interfaz agregados completos
b5b01c5 | 2026-06-26 | Diego Castro Satt | verion pre-eliminar de como se veria el conversor
01ff165 | 2026-06-26 | Diego Castro Satt | agrego el resto de unidades para la conversion
```

f5dd394 | 2026-06-26 | Diego Castro Satt | merge de feature-conversiones a main agregando algunas conversiones de unidades
c593b64 | 2026-06-26 | Diego Castro Satt | agrego conversiones y logica de las unidades de masa, temp y longitud
93caef5 | 2026-06-26 | Diego Castro Satt | prueba de validacion
6b8b89e | 2026-06-25 | Diego Castro Satt | agrego diseno a la interfaz de conversiones
9e02af6 | 2026-06-25 | Diego Castro Satt | agrego pruebas de validacion
227ac44 | 2026-06-25 | Diego Castro Satt | agrego al gitignore archivo de mac DS_store
b185329 | 2026-06-25 | Diego Castro Satt | agrega archivos de validaciones de no permitido numero negativos
25e5e6c | 2026-06-25 | Diego Castro Satt | agregando unidades de masa y temperatura al html
7f32572 | 2026-06-24 | Diego Castro Satt | primer codigo html para visualizacion de software
fafd46b | 2026-06-24 | Diego Castro Satt | archivos base para minisoftware de conversiones

Los días que hubo avance en el desarrollo son 7, los cuales son:

2026-06-24
2026-06-25
2026-06-26
2026-06-28
2026-06-29
2026-06-30
2026-07-01

Las ramas ocupadas en este proyecto fueron 4, las cuales son:

1. Branch main: Es la rama principal del proyecto. Aquí está la versión final del conversor.
2. Branch feature-conversiones: Es la rama que se utilizó para desarrollar la lógica de conversiones, validaciones y fórmulas.
3. Branch feature-interfaz: Es la rama encargada de mejorar la interfaz visual del conversor, como de los estilos, los colores, entre otros elementos visuales.
4. Branch test: Es la rama responsable de agregar y ejecutar pruebas para comprobar que algunas conversiones funcionen correctamente.

Los archivos más modificados fueron el README.md y el conversiones.html

6 README.md
6 conversor.html
5 Src/softwaremini.js
4 Src/unidades.js
4 Src/conversiones.js
3 Test/prueba.js
3 Style/conversor.css
3 .gitignore
2 Test/pruebas.js
1 ConversionInforme.docx

Esto nos podría indicar que los archivos README.md y conversiones.html se podrían ver como hotspots.

Los commits más relevantes que se pueden observar son:

01ff165 agrego el resto de unidades para la conversion, que afecto al archivo conversiones.js
acf9217 cambio de consumo de bencina a monedas de todos los países, que afecto al archivo unidades.js
bf6032c Separacion de conversiones por tipo y categoría, que afecto a los archivos softwaremini.js y conversor.html ya que se redujo la cantidad de líneas considerablemente para la mejora del UX del software.

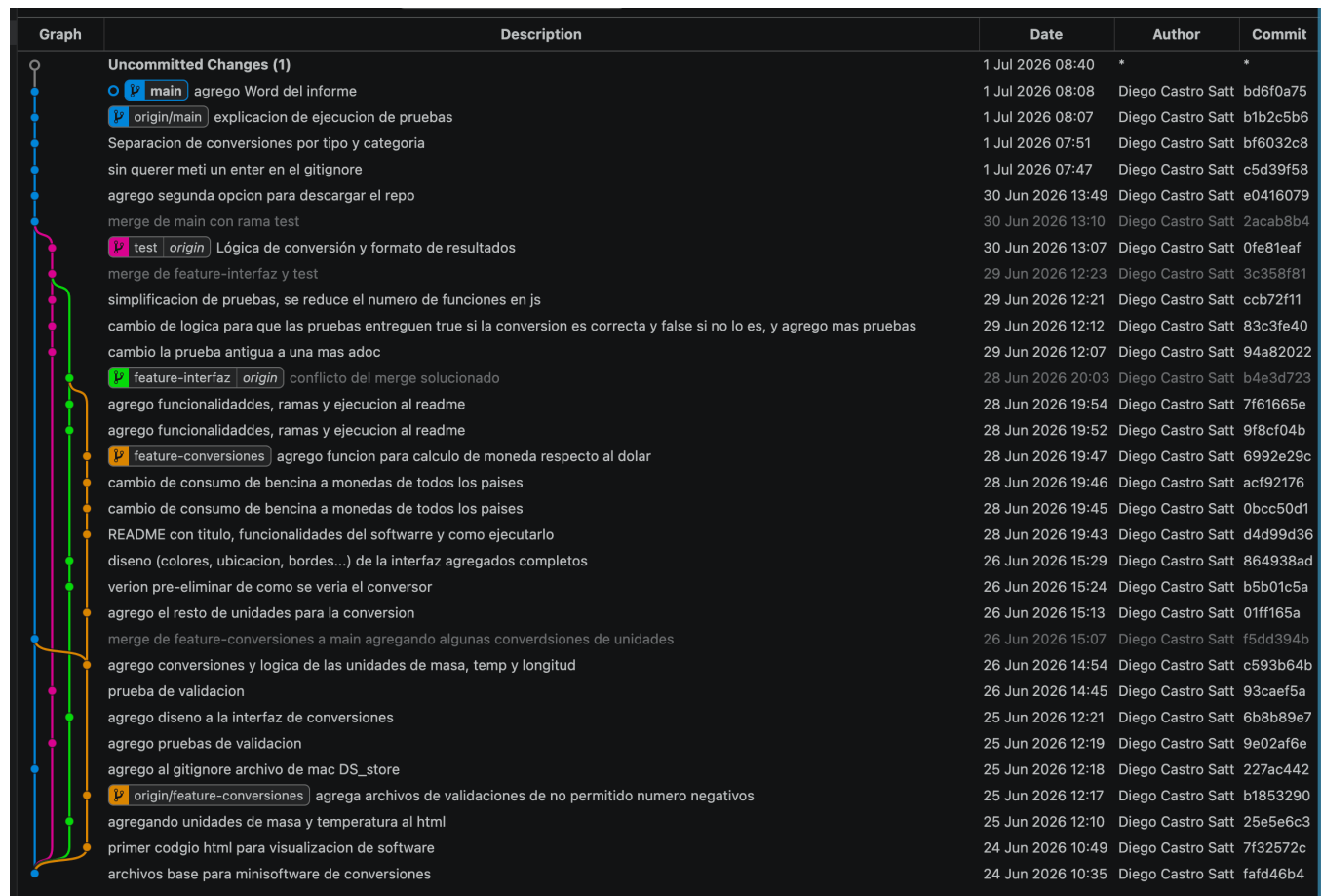
Los merges realizados están en los commits:

acab8b merge de main con rama test
3c358f8 merge de feature-interfaz y test
b4e3d72 (origin/feature-interfaz, feature-interfaz) conflicto del merge solucionado
f5dd394 merge de feature-conversiones a main agregando algunas conversiones de unidades

En la siguiente imagen se presenta el Git Graph, que corresponde a una representación gráfica de la salida del comando `git log --graph --oneline --all`, permitiendo visualizar de manera más clara el historial de commits, las ramas y los merges del proyecto.

La Figura 1 ilustra el historial de Git a través de las ramas `main`, `feature-conversiones`, `feature-interfaz` y `test`. El gráfico evidencia un desarrollo en paralelo integrado mediante `three-way merge`, lo que conservó el historial completo. Esto demuestra un uso de trabajo en Git, facilitando la independencia de tareas y la resolución ordenada de los conflictos.

Fig. 1 Gráfico con Git Graph



En la imagen se observa el historial del repositorio con las distintas ramas del proyecto (`main`, `feature-conversiones`, `feature-interfaz` y `test`), mostrando la evolución del desarrollo, la integración de cambios mediante `three-way merge` y la conservación del historial de los commits.

Los principales merges realizados fueron:

f5dd394: Merge de la rama `feature-conversiones` hacia `main`.

3c358f8: Merge de la rama `feature-interfaz` con `test`.

2acab8b: Merge de la rama `test` hacia `main`.

El Conflicto se generó en el archivo `README.md` entre las ramas `feature-interfaz` y `feature-conversiones`.

```
<<<<<<< HEAD
## Tecnologías utilizadas
- HTML
- CSS
- JavaScript
- git
- GitHub
=====
```

```
## Tecnologías utilizadas
- html
- css
- JavaScript
- git
- GitHub
>>>>>> feature-interfaz
<<<<<< HEAD
- Validación de datos ingresados.
=====
- Validación de datos ingresados por el usuario.
>>>>>> feature-interfaz
```

El conflicto se produjo en el archivo README.md, específicamente en la sección “Funcionalidades” y “Tecnologías utilizadas”. La rama HEAD tenía la lista con nombres escritos con mayúscula inicial, mientras que feature-interfaz tenía algunos nombres en minúscula y en la rama HEAD estaba escrito “Validación de datos ingresados” y en feature-interfaz estaba escrito “Validación de datos ingresados por el usuario”. La resolución manual consistió en unificar el formato, dejando todos los nombres como inicialmente estaba en feature-interfaz e incorporando la parte de “por el usuario” a ambas ramas. Cerrando este merge con el commit b4e3d72 - merge con conflicto solucionado

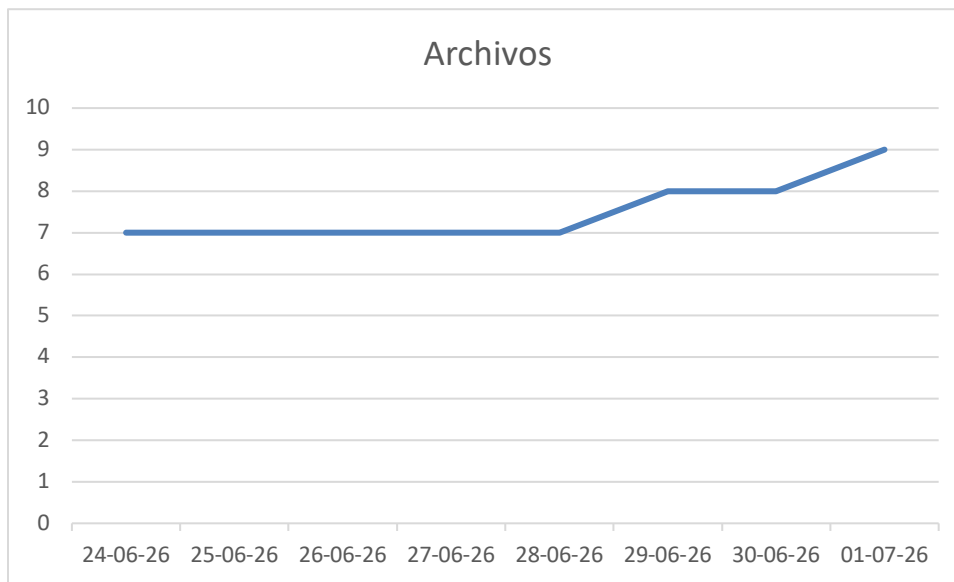
Los archivos a lo largo del desarrollo del proyecto tuvieron varias inserciones (líneas agregadas) y varias líneas eliminadas para así poder llegar a el software finalizado.

Tabla. 1 Tabla de inserciones y eliminaciones por archivo

Archivo	Líneas agregadas (Inserts)	Líneas eliminadas (Deletes)
.gitignore	2	1
README.md	105	2
Src/conversiones.js	96	36
Src/unidades.js	332	10
Src/softwaremini.js	156	14
Style/conversor.css	93	12
Test/prueba.js	83	34
Test/pruebas.js	7	(archivo eliminado)
conversor.html	474	379

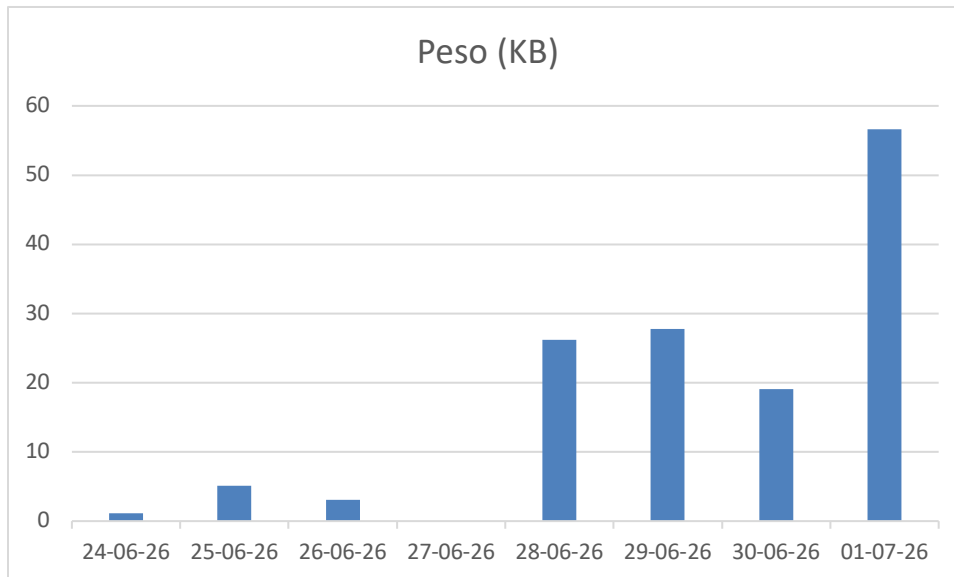
La Tabla 1 muestra las líneas agregadas y eliminadas por archivo. conversor.html fue el más modificado (474 inserciones y 379 eliminaciones) debido al refinamiento continuo de la interfaz. Por otra parte, Src/unidades.js sumó múltiples categorías de unidades y monedas, mientras que README.md y Test/prueba.js se reflejan la actualización de la documentación y las pruebas.

Fig. 2 Gráfico realizado con Excel y Git de la cantidad de archivos por día



La Figura 2 muestra la evolución de los archivos en el proyecto. La estructura se mantuvo con siete archivos entre el 24 y 28 de junio, luego, creció a ocho el 29 de junio tras añadir las pruebas, y llegó a nueve el 1 de julio con la agregación del informe.

Fig. 3 Gráfico realizado con Excel y Git de la cantidad de peso del proyecto por día



La Figura 3 nos muestra el crecimiento en kilobytes del proyecto. Los incrementos del 28 y 29 de junio responden a nuevas conversiones, mejoras de interfaz y documentación. Sin embargo, el aumento más drástico ocurrió el 1 de julio, duplicando el tamaño del repositorio por el informe y archivos finales. Este incremento es consistente con el avance del desarrollo.

Reflexión final

Este proyecto me permitió aprender el uso práctico de Git y GitHub para el control de versiones. Entendí la importancia de realizar commits claros, trabajar con ramas para desarrollar funcionalidades de forma independiente y utilizar merges para integrar los cambios. Además, aprendí a resolver conflictos de manera manual y a interpretar el historial del repositorio para analizar la evolución del proyecto. Finalmente, comprobé que un desarrollo incremental facilita la organización del trabajo, agregar nuevas funcionalidades de forma progresiva y mantener un software más ordenado. Durante el desarrollo del proyecto se utilizó Codex como apoyo para reorganizar parte del código, eliminando código repetitivo o innecesario con el fin de mejorar la estructura del proyecto. También se utilizó como apoyo en la implementación de la integración de monedas y en la redacción de algunas secciones de la documentación del informe (en el peso en kb del proyecto y en el conteo de líneas agregadas y eliminadas por archivo). Todas las decisiones fueron revisadas y comprendidas antes de incorporarlas al proyecto.